Explicación del desarrollo

Aplicación de alquiler de scooters

Elaborado por: Henry Antonio Mendoza Puerta.

Índice

Haz clic en una sección para ir a ella. Al final de cada sección hay un enlace para volver aquí.

- 1. Enunciado
- 2. Funcionalidades
- 3. Interfaz de la aplicación
- 4. Patrones a utilizar y por qué
- 5. Diagramas
- 6. Estructura del proyecto
- Paso 1. Crear el proyecto Maven
- Paso 2. Modelo
- Paso 3. Builder
- Paso 4. Repository
- Paso 5. Adapter
- Paso 6. Service
- Paso 7. Controller
- Paso 8. Singleton
- Paso 9. Vista (JFrame)
- Paso 10. Main y ejecución
- Paso 11. Probar la aplicación

1. Enunciado

Una empresa de alquiler de scooters eléctricos necesita una aplicación de escritorio para gestionar su flota, los alquileres de sus clientes y los pagos. La aplicación debe desarrollarse en Java con Apache NetBeans, como proyecto Maven, y con una interfaz Swing basada en JFrame que pueda editarse tanto en la vista Source como en la vista Design.

La aplicación debe permitir registrar, listar, buscar por ID, filtrar, actualizar y eliminar scooters y alquileres, además de registrar los pagos y consultarlos. Cada alquiler guarda su fecha y hora de inicio y de fin con LocalDateTime, y su monto se calcula como los minutos por la tarifa por minuto del scooter. El pago se realiza con una tarjeta de 16 dígitos mediante una pasarela de pago externa. Un alquiler pagado no puede modificarse y un scooter que tiene alquileres no puede eliminarse.

El modelo debe tener clases asociadas: Scooter, Alquiler (que tiene un Scooter) y Pago (que pertenece a un Alquiler). El código se organiza en paquetes por capa (model, repository, service, controller y view), con un repositorio genérico CRUD cuya implementación guarda los datos en una lista en memoria. Deben aplicarse los patrones MVC, Singleton, Builder, Adapter y Repository, y presentarse dos diagramas: el de clases del dominio y el de arquitectura.

Esta guía es un ejemplo resuelto de este enunciado. Úsala como referencia para aplicar lo mismo en tu propio trabajo: cambia el tema, pero conserva la estructura, los patrones y los diagramas.

Volver al índice

2. Funcionalidades

Módulo Scooters

- Registrar un scooter con su marca, modelo y tarifa por minuto.
- Listar todos los scooters.
- Buscar un scooter por su ID.
- Filtrar los scooters por marca.
- Actualizar los datos de un scooter.
- Eliminar un scooter, solo si no tiene alquileres.

Módulo Alquileres

- Registrar un alquiler eligiendo el scooter, el cliente y los minutos.
- Calcular automáticamente la fecha de fin y el monto del alquiler.
- Listar todos los alquileres.
- Buscar un alquiler por su ID.
- Filtrar los alquileres por un rango de fechas.
- Actualizar el cliente y los minutos de un alquiler; la fecha de fin y el monto se recalculan.
- Eliminar un alquiler.

Módulo Pagos

- Pagar un alquiler pendiente con una tarjeta de 16 dígitos.
- Listar los pagos realizados.
- Buscar un pago por su ID.
- Filtrar los pagos por un rango de fechas.

Reglas de la aplicación

- El alquiler empieza en el momento de registrarlo y termina según los minutos indicados.
- Monto = minutos × tarifa por minuto del scooter.
- Si se cambian los minutos de un alquiler, la fecha de fin y el monto se recalculan.
- Un alquiler pagado no se puede modificar ni pagar otra vez.
- Un scooter que tiene alquileres no se puede eliminar.
- La tarjeta debe tener 16 dígitos y del pago solo se guardan los últimos 4 (**** 1234).

Volver al índice

3. Interfaz de la aplicación

Así se verá la aplicación en cada funcionalidad. Lo escrito en los campos es un ejemplo de ingreso; las tablas muestran datos ya registrados.

Menú principal

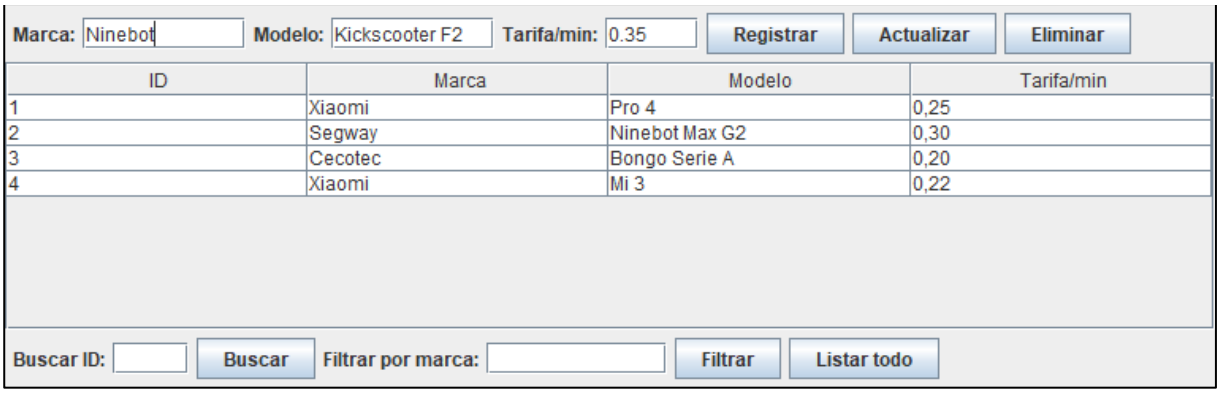
En esta vista debemos elegir el módulo que queremos gestionar: scooters, alquileres o pagos.



Scooters: registrar

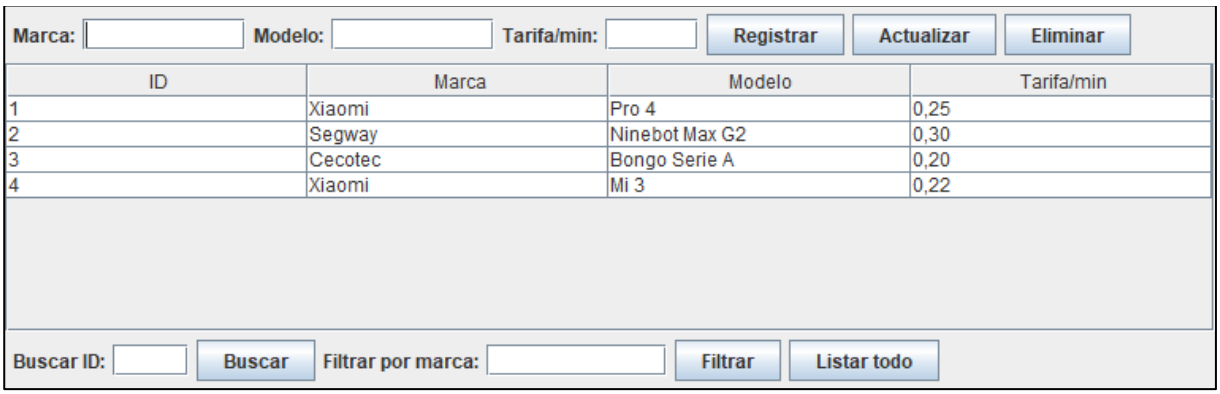
En esta vista debemos registrar un scooter: escribimos la marca, el modelo y la tarifa por minuto, y pulsamos Registrar.

Ejemplo de ingreso: Marca Ninebot, Modelo Kickscooter F2, Tarifa/min 0.35.



Scooters: listar

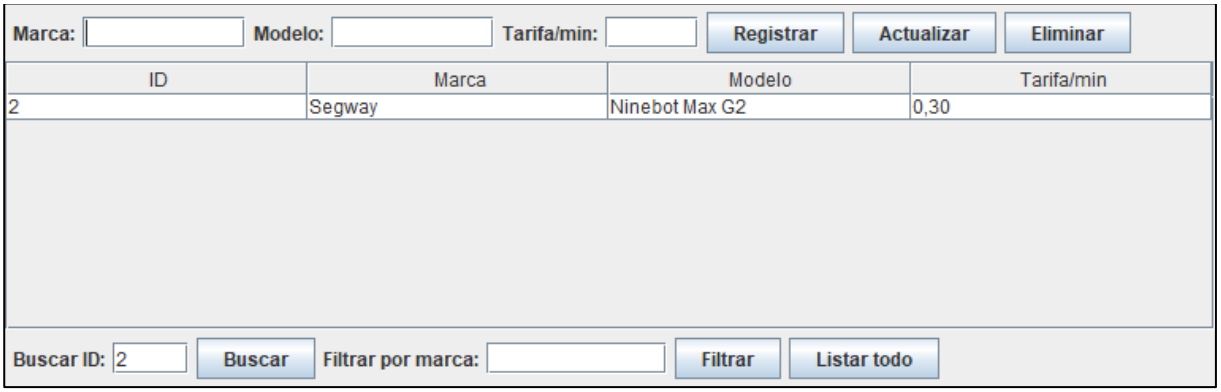
En esta vista debemos listar todos los scooters: pulsamos Listar todo.



Scooters: buscar por ID

En esta vista debemos buscar un scooter por su ID: escribimos el ID y pulsamos Buscar.

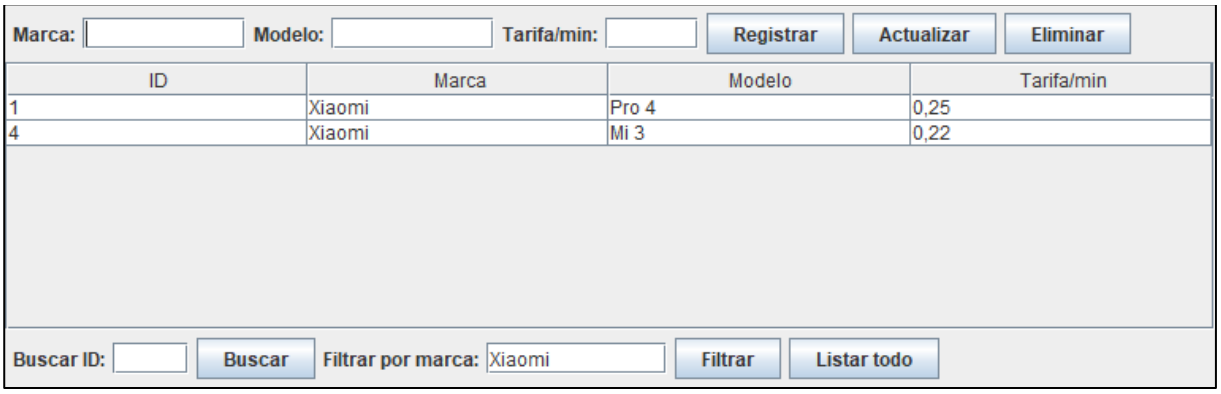
Ejemplo de ingreso: Buscar ID 2.



Scooters: filtrar por marca

En esta vista debemos filtrar los scooters por marca: escribimos la marca y pulsamos Filtrar.

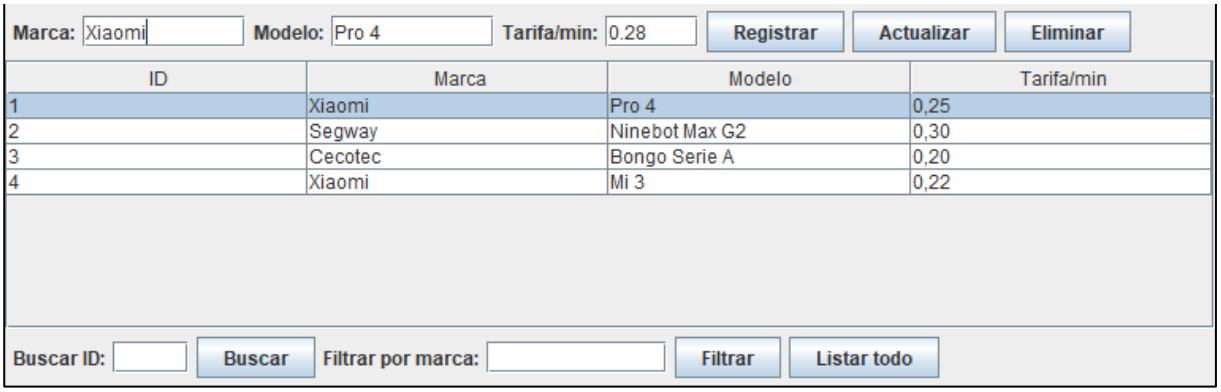
Ejemplo de ingreso: Filtrar por marca Xiaomi.



Scooters: actualizar

En esta vista debemos actualizar un scooter: elegimos su fila en la tabla, cambiamos los datos y pulsamos Actualizar.

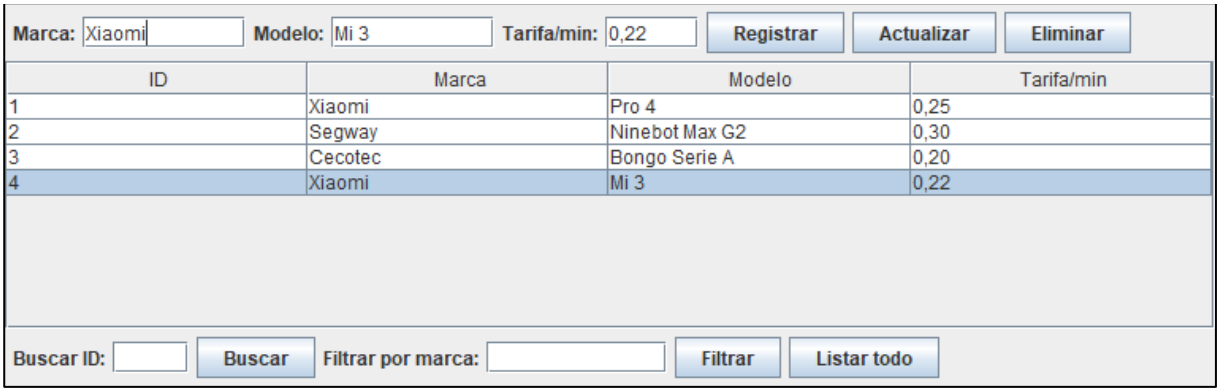
Ejemplo de ingreso: Fila 1 (Xiaomi Pro 4): Tarifa/min de 0.25 a 0.28.



Scooters: eliminar

En esta vista debemos eliminar un scooter: elegimos su fila y pulsamos Eliminar. Si el scooter tiene alquileres, no se puede eliminar.

Ejemplo de ingreso: Fila 4 (Xiaomi Mi 3), que no tiene alquileres.



Alquileres: registrar

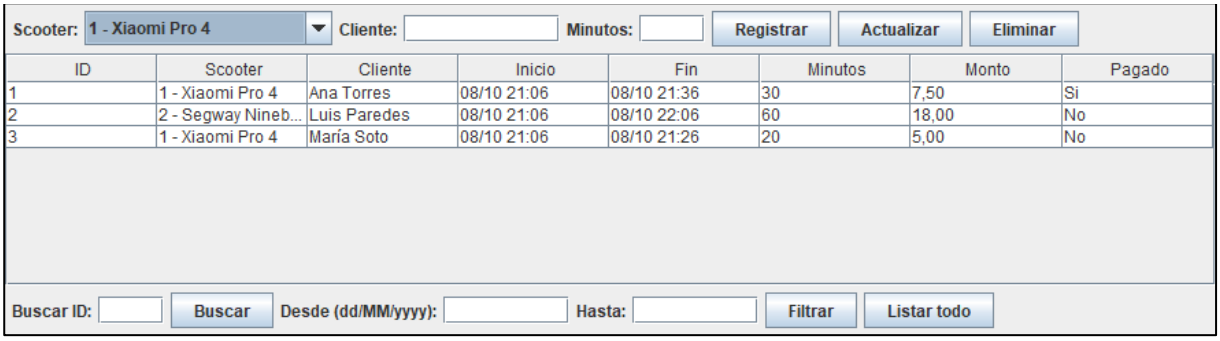
En esta vista debemos registrar un alquiler: elegimos el scooter, escribimos el cliente y los minutos, y pulsamos Registrar. El monto se calcula solo.

Ejemplo de ingreso: Scooter 3 - Cecotec Bongo Serie A, Cliente Carlos Ruiz, Minutos 45. Monto: 45 × 0.20 = 9,00.



Alquileres: listar

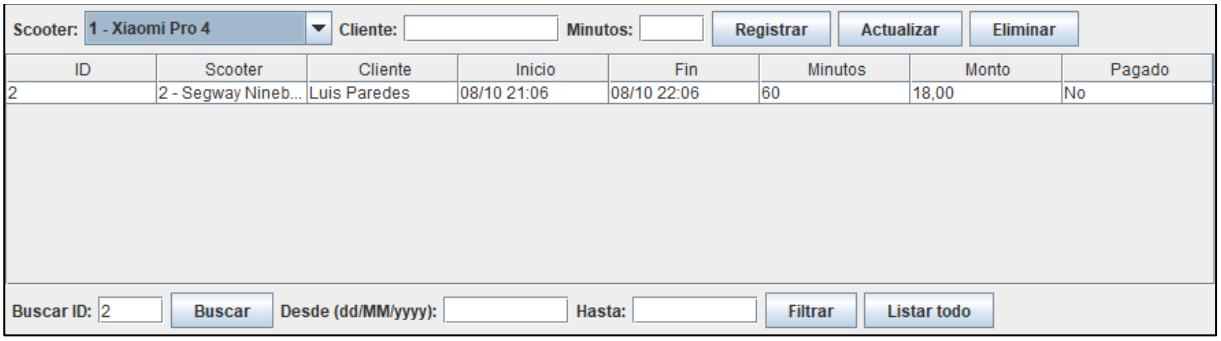
En esta vista debemos listar todos los alquileres: pulsamos Listar todo.



Alquileres: buscar por ID

En esta vista debemos buscar un alquiler por su ID: escribimos el ID y pulsamos Buscar.

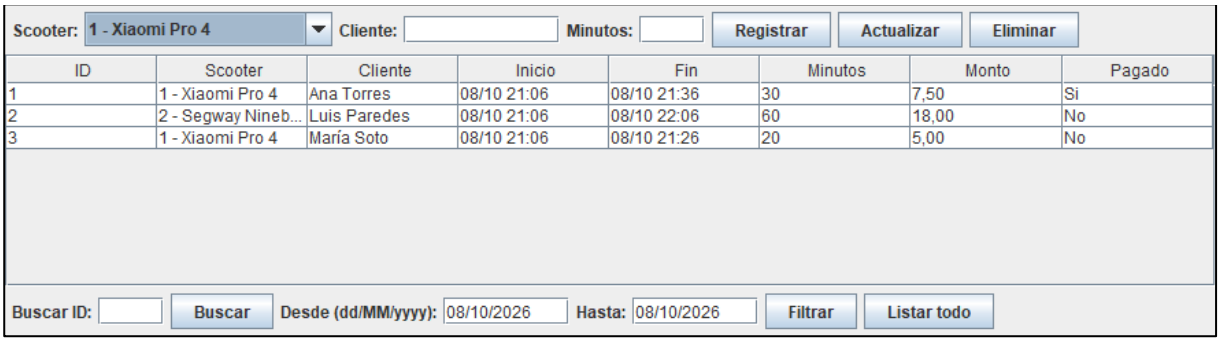
Ejemplo de ingreso: Buscar ID 2.



Alquileres: filtrar por fechas

En esta vista debemos filtrar los alquileres por rango de fechas: escribimos la fecha de inicio y la de fin, y pulsamos Filtrar.

Ejemplo de ingreso: Desde y Hasta con la fecha de hoy, en formato dd/MM/yyyy.



Alquileres: actualizar

En esta vista debemos actualizar un alquiler: elegimos su fila, cambiamos el cliente o los minutos y pulsamos Actualizar. Un alquiler pagado no se puede actualizar.

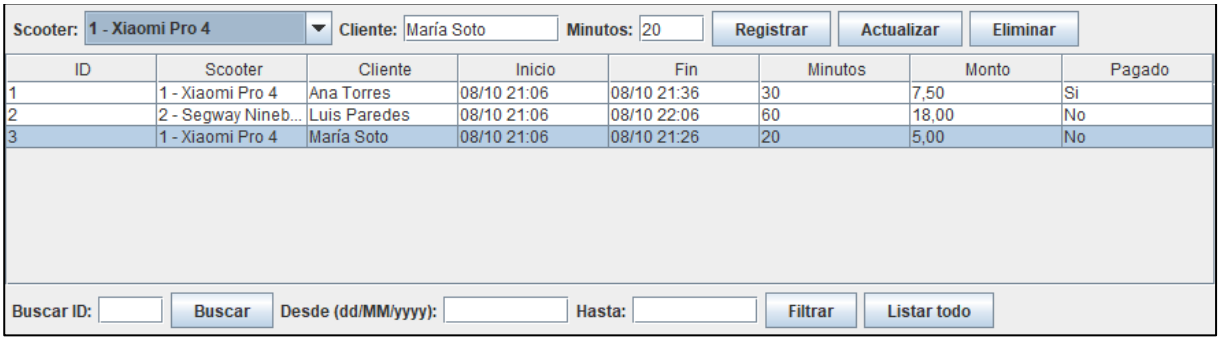
Ejemplo de ingreso: Fila 2 (Luis Paredes): Minutos de 60 a 90.



Alquileres: eliminar

En esta vista debemos eliminar un alquiler: elegimos su fila y pulsamos Eliminar.

Ejemplo de ingreso: Fila 3 (María Soto).



Pagos: pagar

En esta vista debemos pagar un alquiler pendiente: elegimos el alquiler, escribimos la tarjeta de 16 dígitos y pulsamos Pagar.

Ejemplo de ingreso: Alquiler 2 - Luis Paredes - 18,00, Tarjeta 4111111111115678. Del pago solo se guarda **** 5678.



Pagos: listar

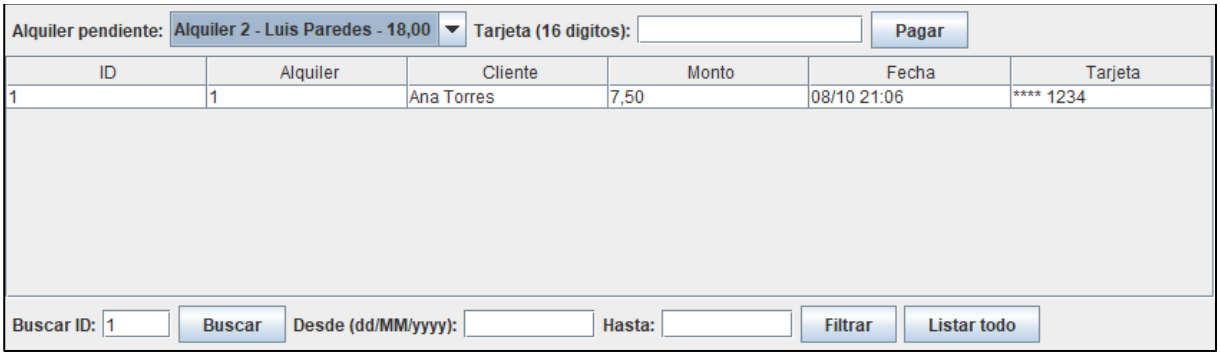
En esta vista debemos listar todos los pagos: pulsamos Listar todo.



Pagos: buscar por ID

En esta vista debemos buscar un pago por su ID: escribimos el ID y pulsamos Buscar.

Ejemplo de ingreso: Buscar ID 1.



Pagos: filtrar por fechas

En esta vista debemos filtrar los pagos por rango de fechas: escribimos la fecha de inicio y la de fin, y pulsamos Filtrar.

Ejemplo de ingreso: Desde y Hasta con la fecha de hoy, en formato dd/MM/yyyy.



Volver al índice

4. Patrones a utilizar y por qué

Patrón	Tipo	Dónde	Por qué
MVC	Arquitectura	view, controller, model	Separa las ventanas, el control y los datos.
Singleton	Creacional	Contexto	Una sola instancia crea y comparte los objetos de la aplicación.
Builder	Creacional	ScooterBuilder, AlquilerBuilder, PagoBuilder	Construye objetos validando sus datos en un solo lugar.
Adapter	Estructural	PasarelaPagoAdapter	Permite usar la librería de pago externa sin cambiar nuestro código.
Repository	Acceso a datos	CrudRepository, interfaces e Impl	Los services no dependen de dónde se guardan los datos.

Volver al índice

5. Diagramas

En tu trabajo debes presentar estos dos diagramas:

Diagrama	Qué debe mostrar
Diagrama de clases del dominio	Solo las clases del modelo del dominio, con sus atributos, métodos y relaciones. No incluye ventanas, controllers, services ni repositorios.
Diagrama de arquitectura	Desde la vista, cómo se conectan los componentes y en qué lugar está cada patrón. Con una leyenda separada por grupos (patrones y tipos de relación).

¿Qué es el dominio?

El dominio representa el problema real que resuelve la aplicación; por eso, al tratarse del alquiler de scooters, el dominio está representado por las clases Scooter, Alquiler y Pago.

Figura 1. Diagrama de clases del dominio (solo Scooter, Alquiler y Pago).

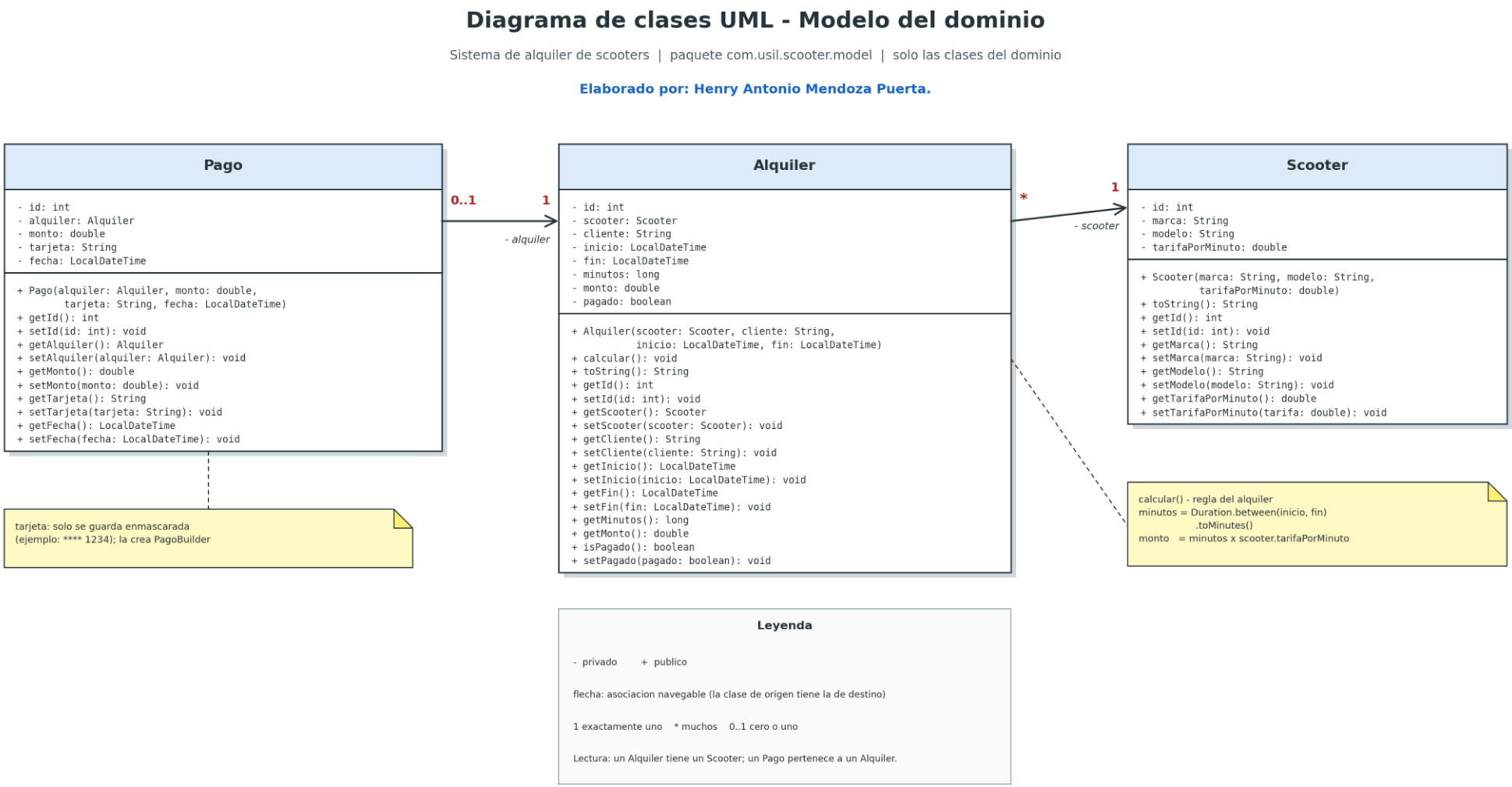
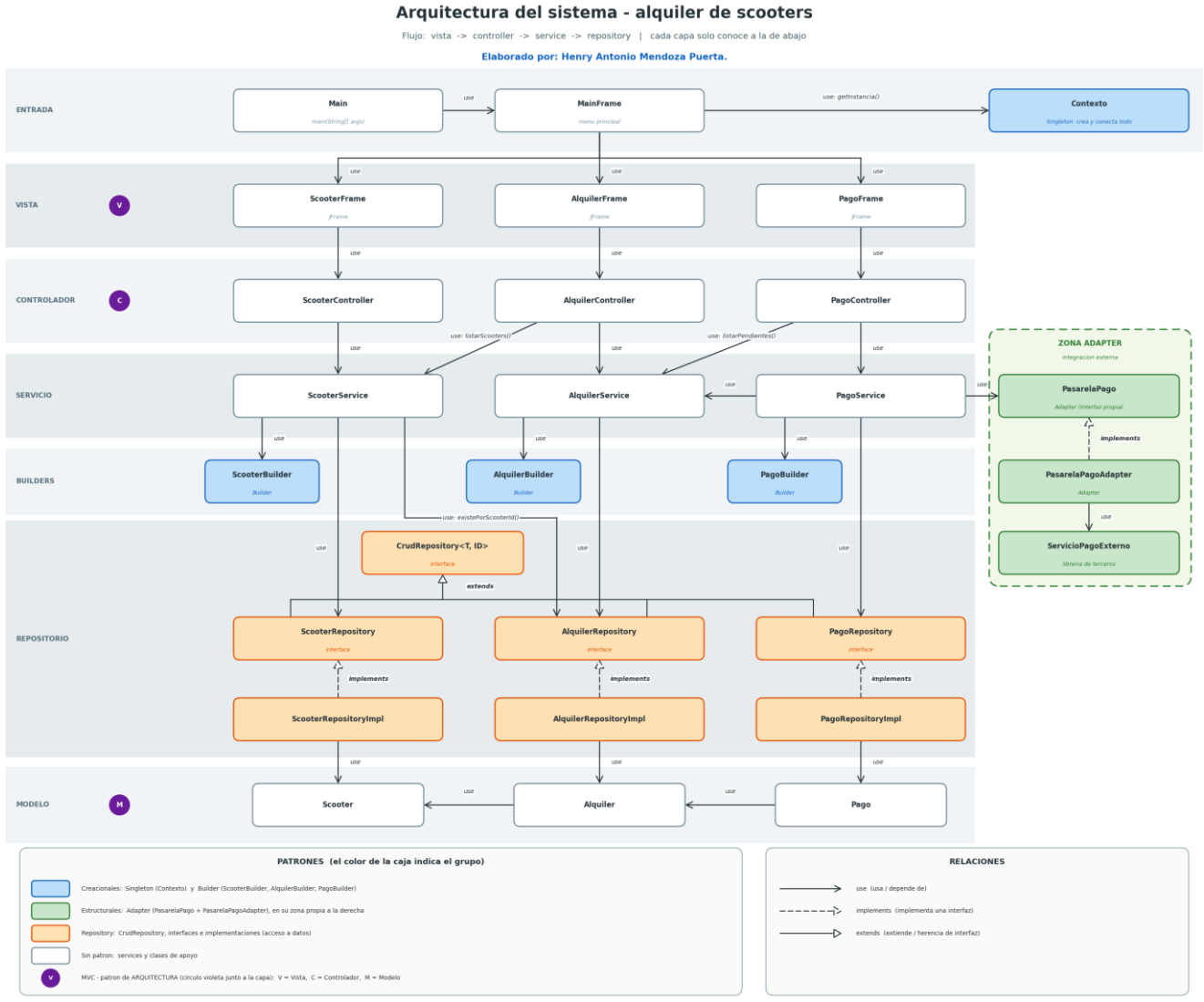


Figura 2. Arquitectura de la aplicación: capas, patrones y relaciones (use, extends, implements).



Volver al índice

6. Estructura del proyecto

Flujo: vista → controller → service → repository.

```
ScooterRental/
  pom.xml
  README.md
  PATRONES.md
  docs/ (diagramas .png y .drawio, esta guía)
  src/main/java/com/usil/scooter/
    adapter/
      PasarelaPago.java
      PasarelaPagoAdapter.java
      ServicioPagoExterno.java
    builder/
      AlquilerBuilder.java
      PagoBuilder.java
      ScooterBuilder.java
    controller/
      AlquilerController.java
      PagoController.java
      ScooterController.java
    model/
      Alquiler.java
      Pago.java
      Scooter.java
    repository/
      impl/
        AlquilerRepositoryImpl.java
        PagoRepositoryImpl.java
        ScooterRepositoryImpl.java
      AlquilerRepository.java
      CrudRepository.java
      PagoRepository.java
      ScooterRepository.java
    service/
      AlquilerService.java
      PagoService.java
      ScooterService.java
    singleton/
      Contexto.java
    view/
      AlquilerFrame.form
      AlquilerFrame.java
      MainFrame.form
      MainFrame.java
      PagoFrame.form
      PagoFrame.java
      ScooterFrame.form
      ScooterFrame.java
    Main.java
```

Volver al índice

Paso 1. Crear el proyecto Maven

En NetBeans: File > New Project > Java with Maven > Java Application. Usa groupId com.usil y artifactId ScooterRental.

Reemplaza el pom.xml por este:

Archivo: pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.usil</groupId>
    <artifactId>ScooterRental</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>
```

```
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <exec.mainClass>com.usil.scooter.Main</exec.mainClass>
    </properties>
</project>
```

Crea el paquete com.usil.scooter y dentro estos: model, builder, repository, repository.impl, adapter, service, controller, singleton y view.

Volver al índice

Paso 2. Modelo

Clases simples con atributos privados, getters y setters. Alquiler tiene un Scooter y calcula minutos y monto. Pago pertenece a un Alquiler.

Archivo: src/main/java/com/usil/scooter/model/Scooter.java

```
package com.usil.scooter.model;

public class Scooter {

    private int id;
    private String marca;
    private String modelo;
    private double tarifaPorMinuto;

    public Scooter(String marca, String modelo, double tarifaPorMinuto) {
        this.marca = marca;
        this.modelo = modelo;
        this.tarifaPorMinuto = tarifaPorMinuto;
    }

    public int getId() { return id; }

    public void setId(int id) { this.id = id; }

    public String getMarca() { return marca; }

    public void setMarca(String marca) { this.marca = marca; }

    public String getModelo() { return modelo; }

    public void setModelo(String modelo) { this.modelo = modelo; }

    public double getTarifaPorMinuto() { return tarifaPorMinuto; }

    public void setTarifaPorMinuto(double tarifaPorMinuto) { this.tarifaPorMinuto =
tarifaPorMinuto; }

    @Override
    public String toString() {
        return id + " - " + marca + " " + modelo;
    }
}
```

Archivo: src/main/java/com/usil/scooter/model/Alquiler.java

```
package com.usil.scooter.model;

import java.time.Duration;
import java.time.LocalDateTime;

public class Alquiler {

    private int id;
    private Scooter scooter;
    private String cliente;
    private LocalDateTime inicio;
```

```
    private LocalDateTime fin;
    private long minutos;       // se calcula con calcular()
    private double monto;       // se calcula con calcular()
    private boolean pagado;

    public Alquiler(Scooter scooter, String cliente, LocalDateTime inicio, LocalDateTime fin) {
        this.scooter = scooter;
        this.cliente = cliente;
        this.inicio = inicio;
        this.fin = fin;
    }

    public int getId() { return id; }

    public void setId(int id) { this.id = id; }

    public Scooter getScooter() { return scooter; }

    public void setScooter(Scooter scooter) { this.scooter = scooter; }

    public String getCliente() { return cliente; }

    public void setCliente(String cliente) { this.cliente = cliente; }

    public LocalDateTime getInicio() { return inicio; }

    public void setInicio(LocalDateTime inicio) { this.inicio = inicio; }

    public LocalDateTime getFin() { return fin; }

    public void setFin(LocalDateTime fin) { this.fin = fin; }

    public long getMinutos() { return minutos; }

    public double getMonto() { return monto; }

    public boolean isPagado() { return pagado; }

    public void setPagado(boolean pagado) { this.pagado = pagado; }

    /** CALCULOS: minutos = Duration entre inicio y fin; monto = minutos x tarifa por minuto del
scooter. */
    public void calcular() {
        minutos = Duration.between(inicio, fin).toMinutes();
        monto = minutos * scooter.getTarifaPorMinuto();
    }

    @Override
    public String toString() {
        return "Alquiler " + id + " - " + cliente + " - " + String.format("%.2f", monto);
    }
}
```

Archivo: src/main/java/com/usil/scooter/model/Pago.java

```
package com.usil.scooter.model;

import java.time.LocalDateTime;

/** Registro de un pago aprobado de un alquiler. */
public class Pago {

    private int id;
    private Alquiler alquiler;
    private double monto;
    private String tarjeta;          // enmascarada: **** 1234
    private LocalDateTime fecha;

    public Pago(Alquiler alquiler, double monto, String tarjeta, LocalDateTime fecha) {
        this.alquiler = alquiler;
        this.monto = monto;
        this.tarjeta = tarjeta;
        this.fecha = fecha;
    }

    public int getId() { return id; }

    public void setId(int id) { this.id = id; }
```

```
    public Alquiler getAlquiler() { return alquiler; }

    public void setAlquiler(Alquiler alquiler) { this.alquiler = alquiler; }

    public double getMonto() { return monto; }

    public void setMonto(double monto) { this.monto = monto; }

    public String getTarjeta() { return tarjeta; }

    public void setTarjeta(String tarjeta) { this.tarjeta = tarjeta; }

    public LocalDateTime getFecha() { return fecha; }

    public void setFecha(LocalDateTime fecha) { this.fecha = fecha; }
}
```

Volver al índice

Paso 3. Builder

Cada builder valida los datos y construye su objeto con llamadas encadenadas. Las validaciones quedan en un solo lugar.

Archivo: src/main/java/com/usil/scooter/builder/ScooterBuilder.java

```
package com.usil.scooter.builder;

import com.usil.scooter.model.Scooter;

public class ScooterBuilder {

    private String marca;
    private String modelo;
    private double tarifaPorMinuto;

    public ScooterBuilder marca(String marca) { this.marca = marca; return this; }

    public ScooterBuilder modelo(String modelo) { this.modelo = modelo; return this; }

    public ScooterBuilder tarifaPorMinuto(double tarifaPorMinuto) { this.tarifaPorMinuto =
tarifaPorMinuto; return this; }

    public Scooter build() {
        if (marca == null || marca.isBlank() || modelo == null || modelo.isBlank()) {
            throw new IllegalArgumentException("Marca y modelo son obligatorios.");
        }
        if (tarifaPorMinuto <= 0) {
            throw new IllegalArgumentException("La tarifa por minuto debe ser mayor que cero.");
        }
        return new Scooter(marca.trim(), modelo.trim(), tarifaPorMinuto);
    }
}
```

Archivo: src/main/java/com/usil/scooter/builder/AlquilerBuilder.java

```
package com.usil.scooter.builder;

import com.usil.scooter.model.Alquiler;
import com.usil.scooter.model.Scooter;
import java.time.LocalDateTime;

public class AlquilerBuilder {

    private Scooter scooter;
    private String cliente;
    private long minutos;

    public AlquilerBuilder scooter(Scooter scooter) { this.scooter = scooter; return this; }

    public AlquilerBuilder cliente(String cliente) { this.cliente = cliente; return this; }
```

```
    public AlquilerBuilder minutos(long minutos) { this.minutos = minutos; return this; }

    public Alquiler build() {
        if (scooter == null) {
            throw new IllegalArgumentException("No hay un scooter elegido.");
        }
        if (cliente == null || cliente.isBlank()) {
            throw new IllegalArgumentException("El cliente es obligatorio.");
        }
        if (minutos <= 0) {
            throw new IllegalArgumentException("Los minutos deben ser mayores que cero.");
        }
        LocalDateTime inicio = LocalDateTime.now();
        return new Alquiler(scooter, cliente.trim(), inicio, inicio.plusMinutes(minutos));
    }
}
```

Archivo: src/main/java/com/usil/scooter/builder/PagoBuilder.java

```
package com.usil.scooter.builder;

import com.usil.scooter.model.Alquiler;
import com.usil.scooter.model.Pago;
import java.time.LocalDateTime;

public class PagoBuilder {

    private Alquiler alquiler;
    private String tarjeta;

    public PagoBuilder alquiler(Alquiler alquiler) { this.alquiler = alquiler; return this; }

    public PagoBuilder tarjeta(String tarjeta) { this.tarjeta = tarjeta; return this; }

    /** El monto sale del alquiler, la fecha es ahora y de la tarjeta solo se guardan los ultimos
4 digitos. */
    public Pago build() {
        if (alquiler == null) {
            throw new IllegalArgumentException("No hay un alquiler para pagar.");
        }
        if (tarjeta == null || tarjeta.trim().length() < 4) {
            throw new IllegalArgumentException("La tarjeta es obligatoria.");
        }
        String digitos = tarjeta.trim();
        String enmascarada = "**** " + digitos.substring(digitos.length() - 4);
        return new Pago(alquiler, alquiler.getMonto(), enmascarada, LocalDateTime.now());
    }
}
```

Volver al índice

Paso 4. Repository

Primero la interfaz genérica CrudRepository. Luego una interfaz por entidad con su consulta propia. Al final las implementaciones, que guardan en una lista.

Archivo: src/main/java/com/usil/scooter/repository/CrudRepository.java

```
package com.usil.scooter.repository;

import java.util.List;
import java.util.Optional;

/** Operaciones generales (CRUD) comunes a todas las entidades. T = entidad, ID = tipo de su
identificador. */
public interface CrudRepository<T, ID> {

    T crear(T entidad);

    Optional<T> buscarPorId(ID id);
```

```
    boolean existePorId(ID id);

    List<T> listar();

    T actualizar(T entidad);

    void eliminar(ID id);
}
```

Archivo: src/main/java/com/usil/scooter/repository/ScooterRepository.java

```
package com.usil.scooter.repository;

import com.usil.scooter.model.Scooter;
import java.util.List;

/** CRUD general + la consulta propia de Scooter. */
public interface ScooterRepository extends CrudRepository<Scooter, Integer> {

    List<Scooter> buscarPorMarca(String marca);
}
```

Archivo: src/main/java/com/usil/scooter/repository/AlquilerRepository.java

```
package com.usil.scooter.repository;

import com.usil.scooter.model.Alquiler;
import java.time.LocalDate;
import java.util.List;

/** CRUD general + las consultas propias de Alquiler. */
public interface AlquilerRepository extends CrudRepository<Alquiler, Integer> {

    /** Alquileres que empezaron entre las dos fechas (incluidas). */
    List<Alquiler> buscarPorFecha(LocalDate desde, LocalDate hasta);

    /** True si algun alquiler usa ese scooter. */
    boolean existePorScooterId(int idScooter);
}
```

Archivo: src/main/java/com/usil/scooter/repository/PagoRepository.java

```
package com.usil.scooter.repository;

import com.usil.scooter.model.Pago;
import java.time.LocalDate;
import java.util.List;

/** CRUD general + la consulta propia de Pago. */
public interface PagoRepository extends CrudRepository<Pago, Integer> {

    /** Pagos realizados entre las dos fechas (incluidas). */
    List<Pago> buscarPorFecha(LocalDate desde, LocalDate hasta);
}
```

Archivo: src/main/java/com/usil/scooter/repository/impl/ScooterRepositoryImpl.java

```
package com.usil.scooter.repository.impl;

import com.usil.scooter.model.Scooter;
import com.usil.scooter.repository.ScooterRepository;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/** Implementa el CRUD y la consulta de ScooterRepository. Los scooters viven en una lista. */
public class ScooterRepositoryImpl implements ScooterRepository {

    private final List<Scooter> scooters = new ArrayList<>();
```

```
    @Override
    public Scooter crear(Scooter scooter) {
        scooter.setId(scooters.stream().mapToInt(Scooter::getId).max().orElse(0) + 1);
        scooters.add(scooter);
        return scooter;
    }

    @Override
    public Optional<Scooter> buscarPorId(Integer id) {
        return scooters.stream().filter(s -> s.getId() == id).findFirst();
    }

    @Override
    public boolean existePorId(Integer id) {
        return scooters.stream().anyMatch(s -> s.getId() == id);
    }

    @Override
    public List<Scooter> listar() {
        return List.copyOf(scooters);
    }

    @Override
    public Scooter actualizar(Scooter scooter) {
        scooters.replaceAll(s -> s.getId() == scooter.getId() ? scooter : s);
        return scooter;
    }

    @Override
    public void eliminar(Integer id) {
        scooters.removeIf(s -> s.getId() == id);
    }

    @Override
    public List<Scooter> buscarPorMarca(String marca) {
        return scooters.stream()
                .filter(s -> s.getMarca().toLowerCase().contains(marca.toLowerCase()))
                .toList();
    }
}
```

Archivo: src/main/java/com/usil/scooter/repository/impl/AlquilerRepositoryImpl.java

```
package com.usil.scooter.repository.impl;

import com.usil.scooter.model.Alquiler;
import com.usil.scooter.repository.AlquilerRepository;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/** Implementa el CRUD y las consultas de AlquilerRepository. Los alquileres viven en una lista.
*/
public class AlquilerRepositoryImpl implements AlquilerRepository {

    private final List<Alquiler> alquileres = new ArrayList<>();

    @Override
    public Alquiler crear(Alquiler alquiler) {
        alquiler.setId(alquileres.stream().mapToInt(Alquiler::getId).max().orElse(0) + 1);
        alquileres.add(alquiler);
        return alquiler;
    }

    @Override
    public Optional<Alquiler> buscarPorId(Integer id) {
        return alquileres.stream().filter(a -> a.getId() == id).findFirst();
    }

    @Override
    public boolean existePorId(Integer id) {
        return alquileres.stream().anyMatch(a -> a.getId() == id);
    }

    @Override
    public List<Alquiler> listar() {
        return List.copyOf(alquileres);
```

```
    }

    @Override
    public Alquiler actualizar(Alquiler alquiler) {
        alquileres.replaceAll(a -> a.getId() == alquiler.getId() ? alquiler : a);
        return alquiler;
    }

    @Override
    public void eliminar(Integer id) {
        alquileres.removeIf(a -> a.getId() == id);
    }

    @Override
    public List<Alquiler> buscarPorFecha(LocalDate desde, LocalDate hasta) {
        return alquileres.stream()
                .filter(a -> !a.getInicio().toLocalDate().isBefore(desde))
                .filter(a -> !a.getInicio().toLocalDate().isAfter(hasta))
                .toList();
    }

    @Override
    public boolean existePorScooterId(int idScooter) {
        return alquileres.stream().anyMatch(a -> a.getScooter().getId() == idScooter);
    }
}
```

Archivo: src/main/java/com/usil/scooter/repository/impl/PagoRepositoryImpl.java

```
package com.usil.scooter.repository.impl;

import com.usil.scooter.model.Pago;
import com.usil.scooter.repository.PagoRepository;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/** Implementa el CRUD de PagoRepository. Los pagos viven en una lista. */
public class PagoRepositoryImpl implements PagoRepository {

    private final List<Pago> pagos = new ArrayList<>();

    @Override
    public Pago crear(Pago pago) {
        pago.setId(pagos.stream().mapToInt(Pago::getId).max().orElse(0) + 1);
        pagos.add(pago);
        return pago;
    }

    @Override
    public Optional<Pago> buscarPorId(Integer id) {
        return pagos.stream().filter(p -> p.getId() == id).findFirst();
    }

    @Override
    public boolean existePorId(Integer id) {
        return pagos.stream().anyMatch(p -> p.getId() == id);
    }

    @Override
    public List<Pago> listar() {
        return List.copyOf(pagos);
    }

    @Override
    public Pago actualizar(Pago pago) {
        pagos.replaceAll(p -> p.getId() == pago.getId() ? pago : p);
        return pago;
    }

    @Override
    public void eliminar(Integer id) {
        pagos.removeIf(p -> p.getId() == id);
    }

    @Override
    public List<Pago> buscarPorFecha(LocalDate desde, LocalDate hasta) {
```

```
        return pagos.stream()
                .filter(p -> !p.getFecha().toLocalDate().isBefore(desde))
                .filter(p -> !p.getFecha().toLocalDate().isAfter(hasta))
                .toList();
    }
}
```

Volver al índice

Paso 5. Adapter

Problema: nuestra aplicación quiere cobrar así: cobrar(monto, tarjeta). Pero la librería de pagos externa funciona distinto y no se puede modificar.

Solución: el adapter implementa nuestra interfaz y, por dentro, llama a la librería. Hace dos traducciones: convierte el monto a céntimos y convierte el número de respuesta en un error entendible.

	Nuestra aplicación (PasarelaPago)	**Librería externa (ServicioPagoExterno)**
Método	cobrar(monto, tarjeta)	procesarTransaccion(tarjeta, centimos)
Monto	15.50 (decimal)	1550 (céntimos, entero)
Respuesta	Si falla, lanza un error con un mensaje.	Devuelve un número: 0 aprobado, 1 tarjeta inválida, 2 monto inválido.

Ejemplo: cobrar(15.50, tarjeta) → el adapter llama procesarTransaccion(tarjeta, 1550). Si la librería responde 1, el adapter lanza el error Tarjeta inválida.

Archivo: src/main/java/com/usil/scooter/adapter/PasarelaPago.java

```
package com.usil.scooter.adapter;

/** Interfaz propia del sistema para cobrar (la que espera el resto del codigo). */
public interface PasarelaPago {

    /** Cobra el monto con la tarjeta. Lanza IllegalArgumentException si el cobro es rechazado. */
    void cobrar(double monto, String tarjeta);
}
```

Archivo: src/main/java/com/usil/scooter/adapter/ServicioPagoExterno.java

```
package com.usil.scooter.adapter;

/**
 * Simula una libreria de TERCEROS que no podemos modificar.
 * Su API es distinta a la nuestra: recibe centimos (long) y responde con un codigo numerico.
 */
public class ServicioPagoExterno {

    /** Devuelve 0 = aprobado, 1 = tarjeta invalida, 2 = monto invalido. */
    public int procesarTransaccion(String numeroTarjeta, long centimos) {
        if (numeroTarjeta == null || numeroTarjeta.length() != 16 ||
!numeroTarjeta.chars().allMatch(Character::isDigit)) {
            return 1;
        }
        if (centimos <= 0) {
            return 2;
        }
        return 0;
    }
}
```

Archivo: src/main/java/com/usil/scooter/adapter/PasarelaPagoAdapter.java

```
package com.usil.scooter.adapter;

/** ADAPTER: convierte nuestra interfaz PasarelaPago a la API del servicio externo. */
public class PasarelaPagoAdapter implements PasarelaPago {

    private final ServicioPagoExterno servicioExterno = new ServicioPagoExterno();

    @Override
    public void cobrar(double monto, String tarjeta) {
        int codigo = servicioExterno.procesarTransaccion(tarjeta.trim(), Math.round(monto * 100));
        if (codigo == 1) {
            throw new IllegalArgumentException("Tarjeta invalida: escriba 16 digitos.");
        }
        if (codigo == 2) {
            throw new IllegalArgumentException("Monto invalido.");
        }
    }
}
```

Volver al índice

Paso 6. Service

Aquí están las reglas de negocio. Cada service recibe sus dependencias por el constructor.

Archivo: src/main/java/com/usil/scooter/service/ScooterService.java

```
package com.usil.scooter.service;

import com.usil.scooter.builder.ScooterBuilder;
import com.usil.scooter.model.Scooter;
import com.usil.scooter.repository.AlquilerRepository;
import com.usil.scooter.repository.ScooterRepository;
import java.util.List;
import java.util.Optional;

public class ScooterService {

    private final ScooterRepository repositorio;
    private final AlquilerRepository alquilerRepositorio;

    public ScooterService(ScooterRepository repositorio, AlquilerRepository alquilerRepositorio) {
        this.repositorio = repositorio;
        this.alquilerRepositorio = alquilerRepositorio;
    }

    public void registrar(String marca, String modelo, double tarifaPorMinuto) {
        repositorio.crear(new
ScooterBuilder().marca(marca).modelo(modelo).tarifaPorMinuto(tarifaPorMinuto).build());
    }

    public List<Scooter> listar() {
        return repositorio.listar();
    }

    public Optional<Scooter> buscarPorId(int id) {
        return repositorio.buscarPorId(id);
    }

    public List<Scooter> filtrarPorMarca(String marca) {
        return repositorio.buscarPorMarca(marca);
    }

    public void actualizar(int id, String marca, String modelo, double tarifaPorMinuto) {
        Scooter scooter = repositorio.buscarPorId(id)
                .orElseThrow(() -> new IllegalArgumentException("No existe el scooter " + id +
".."));
        Scooter nuevo = new
ScooterBuilder().marca(marca).modelo(modelo).tarifaPorMinuto(tarifaPorMinuto).build(); // valida
        scooter.setMarca(nuevo.getMarca());
        scooter.setModelo(nuevo.getModelo());
        scooter.setTarifaPorMinuto(nuevo.getTarifaPorMinuto());
```

```
        repositorio.actualizar(scooter);
    }

    /** Un scooter con alquileres no se puede eliminar. */
    public void eliminar(int id) {
        if (!repositorio.existePorId(id)) {
            throw new IllegalArgumentException("No existe el scooter " + id + ".");
        }
        if (alquilerRepositorio.existePorScooterId(id)) {
            throw new IllegalArgumentException("El scooter tiene alquileres. Elimine primero esos
alquileres.");
        }
        repositorio.eliminar(id);
    }
}
```

Archivo: src/main/java/com/usil/scooter/service/AlquilerService.java

```
package com.usil.scooter.service;

import com.usil.scooter.builder.AlquilerBuilder;
import com.usil.scooter.model.Alquiler;
import com.usil.scooter.model.Scooter;
import com.usil.scooter.repository.AlquilerRepository;
import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

public class AlquilerService {

    private final AlquilerRepository repositorio;

    public AlquilerService(AlquilerRepository repositorio) {
        this.repositorio = repositorio;
    }

    /** El alquiler empieza ahora y termina 'minutos' despues; calcular() obtiene minutos y monto.
*/
    public void registrar(Scooter scooter, String cliente, long minutos) {
        Alquiler alquiler = new
AlquilerBuilder().scooter(scooter).cliente(cliente).minutos(minutos).build();
        alquiler.calcular();
        repositorio.crear(alquiler);
    }

    /** Si cambian los minutos, la fecha de fin, los minutos y el monto se recalculan. */
    public void actualizar(int id, String cliente, long minutos) {
        Alquiler alquiler = repositorio.buscarPorId(id)
                .orElseThrow(() -> new IllegalArgumentException("No existe el alquiler " + id +
"."));
        if (alquiler.isPagado()) {
            throw new IllegalArgumentException("Un alquiler pagado no se puede modificar.");
        }
        Alquiler nuevo = new
AlquilerBuilder().scooter(alquiler.getScooter()).cliente(cliente).minutos(minutos).build(); //
valida
        alquiler.setCliente(nuevo.getCliente());
        alquiler.setFin(alquiler.getInicio().plusMinutes(minutos));
        alquiler.calcular();
        repositorio.actualizar(alquiler);
    }

    /** Lo usa PagoService cuando el cobro fue aprobado. */
    public void marcarPagado(int id) {
        Alquiler alquiler = repositorio.buscarPorId(id)
                .orElseThrow(() -> new IllegalArgumentException("No existe el alquiler " + id +
"."));
        alquiler.setPagado(true);
        repositorio.actualizar(alquiler);
    }

    /** Alquileres que todavia no se pagaron. */
    public List<Alquiler> listarPendientes() {
        return repositorio.listar().stream().filter(a -> !a.isPagado()).toList();
    }

    public List<Alquiler> listar() {
        return repositorio.listar();
```

```
    }

    public Optional<Alquiler> buscarPorId(int id) {
        return repositorio.buscarPorId(id);
    }

    public List<Alquiler> filtrarPorFecha(LocalDate desde, LocalDate hasta) {
        if (desde.isAfter(hasta)) {
            throw new IllegalArgumentException("La fecha de inicio no puede ser posterior a la
fecha de fin.");
        }
        return repositorio.buscarPorFecha(desde, hasta);
    }

    public void eliminar(int id) {
        if (!repositorio.existePorId(id)) {
            throw new IllegalArgumentException("No existe el alquiler " + id + ".");
        }
        repositorio.eliminar(id);
    }
}
```

Archivo: src/main/java/com/usil/scooter/service/PagoService.java

```
package com.usil.scooter.service;

import com.usil.scooter.adapter.PasarelaPago;
import com.usil.scooter.builder.PagoBuilder;
import com.usil.scooter.model.Alquiler;
import com.usil.scooter.model.Pago;
import com.usil.scooter.repository.PagoRepository;
import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

public class PagoService {

    private final PagoRepository repositorio;
    private final PasarelaPago pasarelaPago;
    private final AlquilerService alquilerService;

    public PagoService(PagoRepository repositorio, PasarelaPago pasarelaPago, AlquilerService
alquilerService) {
        this.repositorio = repositorio;
        this.pasarelaPago = pasarelaPago;
        this.alquilerService = alquilerService;
    }

    /** Pagar: cobra con la pasarela (Adapter), guarda el Pago (PagoBuilder) y marca el alquiler
como pagado. */
    public void pagar(int idAlquiler, String tarjeta) {
        Alquiler alquiler = alquilerService.buscarPorId(idAlquiler)
                .orElseThrow(() -> new IllegalArgumentException("No existe el alquiler " +
idAlquiler + "."));
        if (alquiler.isPagado()) {
            throw new IllegalArgumentException("Este alquiler ya esta pagado.");
        }
        pasarelaPago.cobrar(alquiler.getMonto(), tarjeta);
        repositorio.crear(new PagoBuilder().alquiler(alquiler).tarjeta(tarjeta).build());
        alquilerService.marcarPagado(idAlquiler);
    }

    public List<Pago> listar() {
        return repositorio.listar();
    }

    public Optional<Pago> buscarPorId(int id) {
        return repositorio.buscarPorId(id);
    }

    public List<Pago> filtrarPorFecha(LocalDate desde, LocalDate hasta) {
        if (desde.isAfter(hasta)) {
            throw new IllegalArgumentException("La fecha de inicio no puede ser posterior a la
fecha de fin.");
```

```
        }
        return repositorio.buscarPorFecha(desde, hasta);
    }
}
```

Volver al índice

Paso 7. Controller

Reciben los pedidos de la vista y los delegan en los services. Solo conocen services.

Archivo: src/main/java/com/usil/scooter/controller/ScooterController.java

```
package com.usil.scooter.controller;

import com.usil.scooter.model.Scooter;
import com.usil.scooter.service.ScooterService;
import java.util.List;

/** Controller de Scooter. Solo conoce a su service. */
public class ScooterController {

    private final ScooterService scooterService;

    public ScooterController(ScooterService scooterService) {
        this.scooterService = scooterService;
    }

    public void registrar(String marca, String modelo, double tarifa) {
        scooterService.registrar(marca, modelo, tarifa);
    }

    public void actualizar(int id, String marca, String modelo, double tarifa) {
        scooterService.actualizar(id, marca, modelo, tarifa);
    }

    public void eliminar(int id) {
        scooterService.eliminar(id);
    }

    public List<Scooter> listar() {
        return scooterService.listar();
    }

    public List<Scooter> buscar(int id) {
        return scooterService.buscarPorId(id).stream().toList();
    }

    public List<Scooter> filtrar(String marca) {
        return scooterService.filtrarPorMarca(marca);
    }
}
```

Archivo: src/main/java/com/usil/scooter/controller/AlquilerController.java

```
package com.usil.scooter.controller;

import com.usil.scooter.model.Alquiler;
import com.usil.scooter.model.Scooter;
import com.usil.scooter.service.AlquilerService;
import com.usil.scooter.service.ScooterService;
import java.time.LocalDate;
import java.util.List;

/** Controller de Alquiler. Solo conoce a los services. */
public class AlquilerController {

    private final AlquilerService alquilerService;
    private final ScooterService scooterService;

    public AlquilerController(AlquilerService alquilerService, ScooterService scooterService) {
        this.alquilerService = alquilerService;
```

```
        this.scooterService = scooterService;
    }

    public void registrar(Scooter scooter, String cliente, long minutos) {
        alquilerService.registrar(scooter, cliente, minutos);
    }

    public void actualizar(int id, String cliente, long minutos) {
        alquilerService.actualizar(id, cliente, minutos);
    }

    public void eliminar(int id) {
        alquilerService.eliminar(id);
    }

    public List<Alquiler> listar() {
        return alquilerService.listar();
    }

    public List<Alquiler> buscar(int id) {
        return alquilerService.buscarPorId(id).stream().toList();
    }

    public List<Alquiler> filtrar(LocalDate desde, LocalDate hasta) {
        return alquilerService.filtrarPorFecha(desde, hasta);
    }

    public List<Scooter> listarScooters() {
        return scooterService.listar();
    }
}
```

Archivo: src/main/java/com/usil/scooter/controller/PagoController.java

```
package com.usil.scooter.controller;

import com.usil.scooter.model.Alquiler;
import com.usil.scooter.model.Pago;
import com.usil.scooter.service.AlquilerService;
import com.usil.scooter.service.PagoService;
import java.time.LocalDate;
import java.util.List;

/** Controller de Pago. Solo conoce a los services (PagoService y AlquilerService, de donde carga
los alquileres). */
public class PagoController {

    private final PagoService pagoService;
    private final AlquilerService alquilerService;

    public PagoController(PagoService pagoService, AlquilerService alquilerService) {
        this.pagoService = pagoService;
        this.alquilerService = alquilerService;
    }

    public void pagar(int idAlquiler, String tarjeta) {
        pagoService.pagar(idAlquiler, tarjeta);
    }

    public List<Alquiler> alquileresPendientes() {
        return alquilerService.listarPendientes();
    }

    public List<Pago> listar() {
        return pagoService.listar();
    }

    public List<Pago> buscar(int id) {
        return pagoService.buscarPorId(id).stream().toList();
    }

    public List<Pago> filtrar(LocalDate desde, LocalDate hasta) {
        return pagoService.filtrarPorFecha(desde, hasta);
    }
}
```

Volver al índice

Paso 8. Singleton

Una sola instancia crea cada objeto una vez y los conecta por constructor. Simula el contexto de Spring.

Archivo: src/main/java/com/usil/scooter/singleton/Contexto.java

```
package com.usil.scooter.singleton;

import com.usil.scooter.adapter.PasarelaPago;
import com.usil.scooter.adapter.PasarelaPagoAdapter;
import com.usil.scooter.controller.AlquilerController;
import com.usil.scooter.controller.PagoController;
import com.usil.scooter.controller.ScooterController;
import com.usil.scooter.repository.AlquilerRepository;
import com.usil.scooter.repository.PagoRepository;
import com.usil.scooter.repository.ScooterRepository;
import com.usil.scooter.repository.impl.AlquilerRepositoryImpl;
import com.usil.scooter.repository.impl.PagoRepositoryImpl;
import com.usil.scooter.repository.impl.ScooterRepositoryImpl;
import com.usil.scooter.service.AlquilerService;
import com.usil.scooter.service.PagoService;
import com.usil.scooter.service.ScooterService;

/**
 * SINGLETON: una sola instancia que arma el sistema (simula el contexto de Spring).
 * Crea cada objeto una sola vez y las vistas comparten esos mismos objetos.
 */
public class Contexto {

    private static final Contexto INSTANCIA = new Contexto();

    public static Contexto getInstancia() {
        return INSTANCIA;
    }

    private final ScooterRepository scooterRepository = new ScooterRepositoryImpl();
    private final AlquilerRepository alquilerRepository = new AlquilerRepositoryImpl();
    private final PagoRepository pagoRepository = new PagoRepositoryImpl();
    private final PasarelaPago pasarelaPago = new PasarelaPagoAdapter();

    private final ScooterService scooterService = new ScooterService(scooterRepository,
alquilerRepository);
    private final AlquilerService alquilerService = new AlquilerService(alquilerRepository);
    private final PagoService pagoService = new PagoService(pagoRepository, pasarelaPago,
alquilerService);

    private final ScooterController scooterController = new ScooterController(scooterService);
    private final AlquilerController alquilerController = new AlquilerController(alquilerService,
scooterService);
    private final PagoController pagoController = new PagoController(pagoService,
alquilerService);

    private Contexto() {
    }

    public ScooterController scooterController() {
        return scooterController;
    }

    public AlquilerController alquilerController() {
        return alquilerController;
    }

    public PagoController pagoController() {
        return pagoController;
    }
}
```

Volver al índice

Paso 9. Vista (JFrame)

En NetBeans: clic derecho en el paquete view > New > JFrame Form. Arma los componentes en la vista Design y pega el código.

Las ventanas solo conocen a su controller. Cada ventana tiene también un archivo .form que NetBeans crea solo.

Archivo: src/main/java/com/usil/scooter/view/MainFrame.java

```
package com.usil.scooter.view;

import com.usil.scooter.singleton.Contexto;

/** Vista principal: abre las vistas de Scooters y de Alquileres. Se edita en la pestana Design de
NetBeans. */
public class MainFrame extends javax.swing.JFrame {

    public MainFrame() {
        initComponents();
        lblTitulo.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblTitulo.setFont(lblTitulo.getFont().deriveFont(18f));
        setSize(640, 160);
        setLocationRelativeTo(null);
    }

    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    @SuppressWarnings("unchecked")
    private void initComponents() {

        lblTitulo = new javax.swing.JLabel();
        pnlMenu = new javax.swing.JPanel();
        btnScooters = new javax.swing.JButton();
        btnAlquileres = new javax.swing.JButton();
        btnPagos = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Alquiler de Scooters");

        lblTitulo.setText("Alquiler de Scooters");

        pnlMenu.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.CENTER));
        btnScooters.setText("Gestionar scooters");
        btnScooters.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnScootersActionPerformed(evt);
            }
        });

        btnAlquileres.setText("Gestionar alquileres");
        btnAlquileres.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnAlquileresActionPerformed(evt);
            }
        });

        btnPagos.setText("Gestionar pagos");
        btnPagos.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnPagosActionPerformed(evt);
            }
        });

        pnlMenu.add(btnScooters);
        pnlMenu.add(btnAlquileres);
        pnlMenu.add(btnPagos);

        getContentPane().add(lblTitulo, java.awt.BorderLayout.NORTH);
        getContentPane().add(pnlMenu, java.awt.BorderLayout.CENTER);

        pack();
    }// </editor-fold>//GEN-END:initComponents

    private void btnScootersActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnScootersActionPerformed
        new ScooterFrame(Contexto.getInstancia().scooterController()).setVisible(true);
```

```
    }//GEN-LAST:event_btnScootersActionPerformed

    private void btnAlquileresActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnAlquileresActionPerformed
        new AlquilerFrame(Contexto.getInstancia().alquilerController()).setVisible(true);
    }//GEN-LAST:event_btnAlquileresActionPerformed

    private void btnPagosActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnPagosActionPerformed
        new PagoFrame(Contexto.getInstancia().pagoController()).setVisible(true);
    }//GEN-LAST:event_btnPagosActionPerformed

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton btnAlquileres;
    private javax.swing.JButton btnPagos;
    private javax.swing.JButton btnScooters;
    private javax.swing.JLabel lblTitulo;
    private javax.swing.JPanel pnlMenu;
    // End of variables declaration//GEN-END:variables
}
```

Archivo: src/main/java/com/usil/scooter/view/ScooterFrame.java

```
package com.usil.scooter.view;

import com.usil.scooter.controller.ScooterController;
import com.usil.scooter.model.Scooter;
import java.util.List;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

/** Vista de Scooters. Solo conoce a su controller. Se edita en la pestana Design de NetBeans. */
public class ScooterFrame extends javax.swing.JFrame {

    private final ScooterController controlador;
    private final DefaultTableModel modelo = new DefaultTableModel(
            new Object[]{"ID", "Marca", "Modelo", "Tarifa/min"}, 0);

    public ScooterFrame(ScooterController controlador) {
        this.controlador = controlador;
        initComponents();
        tblScooters.setModel(modelo);
        // al elegir una fila, sus datos pasan a los campos para poder actualizar
        tblScooters.getSelectionModel().addListSelectionListener(e -> {
            int fila = tblScooters.getSelectedRow();
            if (!e.getValueIsAdjusting() && fila >= 0) {
                txtMarca.setText(String.valueOf(tblScooters.getValueAt(fila, 1)));
                txtModelo.setText(String.valueOf(tblScooters.getValueAt(fila, 2)));
                txtTarifa.setText(String.valueOf(tblScooters.getValueAt(fila, 3)));
            }
        });
        // los datos son compartidos con la otra vista: al volver a esta ventana se recargan
        addWindowListener(new java.awt.event.WindowAdapter() {
            @Override
            public void windowActivated(java.awt.event.WindowEvent e) {
                mostrar(controlador.listar());
            }
        });
        setSize(780, 420);
        setLocationRelativeTo(null);
        refrescar();
    }

    private void refrescar() {
        txtMarca.setText("");
        txtModelo.setText("");
        txtTarifa.setText("");
        mostrar(controlador.listar());
    }

    private void mostrar(List<Scooter> scooters) {
        modelo.setRowCount(0);
        scooters.forEach(s -> modelo.addRow(new Object[]{s.getId(), s.getMarca(), s.getModelo(),
            String.format("%.2f", s.getTarifaPorMinuto())}));
    }

    /** Ejecuta la accion y, si el controller rechaza los datos, muestra el aviso. */
    private void ejecutar(Runnable accion) {
```

```
        try {
            accion.run();
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Escriba solo numeros en los campos numericos.",
"Aviso", JOptionPane.WARNING_MESSAGE);
        } catch (IllegalArgumentException e) {
            JOptionPane.showMessageDialog(this, e.getMessage(), "Aviso",
JOptionPane.WARNING_MESSAGE);
        }
    }

    /** Devuelve el ID de la fila elegida; si no hay fila elegida lanza un aviso. */
    private int idSeleccionado() {
        int fila = tblScooters.getSelectedRow();
        if (fila < 0) {
            throw new IllegalArgumentException("Seleccione una fila de la tabla.");
        }
        return (Integer) tblScooters.getValueAt(fila, 0);
    }

    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    @SuppressWarnings("unchecked")
    private void initComponents() {

        pnlDatos = new javax.swing.JPanel();
        lblMarca = new javax.swing.JLabel();
        txtMarca = new javax.swing.JTextField();
        lblModelo = new javax.swing.JLabel();
        txtModelo = new javax.swing.JTextField();
        lblTarifa = new javax.swing.JLabel();
        txtTarifa = new javax.swing.JTextField();
        btnRegistrar = new javax.swing.JButton();
        btnActualizar = new javax.swing.JButton();
        btnEliminar = new javax.swing.JButton();
        scrollScooters = new javax.swing.JScrollPane();
        tblScooters = new javax.swing.JTable();
        pnlBuscar = new javax.swing.JPanel();
        lblId = new javax.swing.JLabel();
        txtId = new javax.swing.JTextField();
        btnBuscar = new javax.swing.JButton();
        lblFiltro = new javax.swing.JLabel();
        txtFiltro = new javax.swing.JTextField();
        btnFiltrar = new javax.swing.JButton();
        btnListar = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Scooters");

        pnlDatos.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
        lblMarca.setText("Marca:");

        txtMarca.setColumns(9);

        lblModelo.setText("Modelo:");

        txtModelo.setColumns(9);

        lblTarifa.setText("Tarifa/min:");

        txtTarifa.setColumns(5);

        btnRegistrar.setText("Registrar");
        btnRegistrar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnRegistrarActionPerformed(evt);
            }
        });

        btnActualizar.setText("Actualizar");
        btnActualizar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnActualizarActionPerformed(evt);
            }
        });

        btnEliminar.setText("Eliminar");
        btnEliminar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEliminarActionPerformed(evt);
            }
```

```
        });

        pnlDatos.add(lblMarca);
        pnlDatos.add(txtMarca);
        pnlDatos.add(lblModelo);
        pnlDatos.add(txtModelo);
        pnlDatos.add(lblTarifa);
        pnlDatos.add(txtTarifa);
        pnlDatos.add(btnRegistrar);
        pnlDatos.add(btnActualizar);
        pnlDatos.add(btnEliminar);


        scrollScooters.setViewportView(tblScooters);

        pnlBuscar.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
        lblId.setText("Buscar ID:");

        txtId.setColumns(4);

        btnBuscar.setText("Buscar");
        btnBuscar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnBuscarActionPerformed(evt);
            }
        });

        lblFiltro.setText("Filtrar por marca:");

        txtFiltro.setColumns(10);

        btnFiltrar.setText("Filtrar");
        btnFiltrar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnFiltrarActionPerformed(evt);
            }
        });

        btnListar.setText("Listar todo");
        btnListar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnListarActionPerformed(evt);
            }
        });

        pnlBuscar.add(lblId);
        pnlBuscar.add(txtId);
        pnlBuscar.add(btnBuscar);
        pnlBuscar.add(lblFiltro);
        pnlBuscar.add(txtFiltro);
        pnlBuscar.add(btnFiltrar);
        pnlBuscar.add(btnListar);

        getContentPane().add(pnlDatos, java.awt.BorderLayout.NORTH);
        getContentPane().add(scrollScooters, java.awt.BorderLayout.CENTER);
        getContentPane().add(pnlBuscar, java.awt.BorderLayout.SOUTH);

        pack();
    }// </editor-fold>//GEN-END:initComponents

    private void btnRegistrarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnRegistrarActionPerformed
        ejecutar(() -> {
            controlador.registrar(txtMarca.getText(), txtModelo.getText(),
Double.parseDouble(txtTarifa.getText().replace(',', '.')));
            refrescar();
        });
    }//GEN-LAST:event_btnRegistrarActionPerformed

    private void btnActualizarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnActualizarActionPerformed
        ejecutar(() -> {
            controlador.actualizar(idSeleccionado(), txtMarca.getText(), txtModelo.getText(),
Double.parseDouble(txtTarifa.getText().replace(',', '.')));
            refrescar();
        });
    }//GEN-LAST:event_btnActualizarActionPerformed

    private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnEliminarActionPerformed
```

```
        ejecutar(() -> {
            controlador.eliminar(idSeleccionado());
            refrescar();
        });
    }//GEN-LAST:event_btnEliminarActionPerformed

    private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnBuscarActionPerformed
        ejecutar(() -> mostrar(controlador.buscar(Integer.parseInt(txtId.getText().trim()))));
    }//GEN-LAST:event_btnBuscarActionPerformed

    private void btnFiltrarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnFiltrarActionPerformed
        mostrar(controlador.filtrar(txtFiltro.getText()));
    }//GEN-LAST:event_btnFiltrarActionPerformed

    private void btnListarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnListarActionPerformed
        txtId.setText("");
        txtFiltro.setText("");
        refrescar();
    }//GEN-LAST:event_btnListarActionPerformed

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton btnActualizar;
    private javax.swing.JButton btnBuscar;
    private javax.swing.JButton btnEliminar;
    private javax.swing.JButton btnFiltrar;
    private javax.swing.JButton btnListar;
    private javax.swing.JButton btnRegistrar;
    private javax.swing.JLabel lblFiltro;
    private javax.swing.JLabel lblId;
    private javax.swing.JLabel lblMarca;
    private javax.swing.JLabel lblModelo;
    private javax.swing.JLabel lblTarifa;
    private javax.swing.JPanel pnlBuscar;
    private javax.swing.JPanel pnlDatos;
    private javax.swing.JScrollPane scrollScooters;
    private javax.swing.JTable tblScooters;
    private javax.swing.JTextField txtFiltro;
    private javax.swing.JTextField txtId;
    private javax.swing.JTextField txtMarca;
    private javax.swing.JTextField txtModelo;
    private javax.swing.JTextField txtTarifa;
    // End of variables declaration//GEN-END:variables
}
```

Archivo: src/main/java/com/usil/scooter/view/AlquilerFrame.java

```
package com.usil.scooter.view;

import com.usil.scooter.controller.AlquilerController;
import com.usil.scooter.model.Alquiler;
import com.usil.scooter.model.Scooter;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.List;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

/** Vista de Alquileres. Solo conoce a su controller. Se edita en la pestana Design de NetBeans.
*/
public class AlquilerFrame extends javax.swing.JFrame {

    private static final DateTimeFormatter FORMATO = DateTimeFormatter.ofPattern("dd/MM HH:mm");
    private static final DateTimeFormatter FECHA = DateTimeFormatter.ofPattern("dd/MM/yyyy");

    private final AlquilerController controlador;
    private final DefaultTableModel modelo = new DefaultTableModel(
            new Object[]{"ID", "Scooter", "Cliente", "Inicio", "Fin", "Minutos", "Monto",
"Pagado"}, 0);

    public AlquilerFrame(AlquilerController controlador) {
        this.controlador = controlador;
        initComponents();
        tblAlquileres.setModel(modelo);
        // al elegir una fila, sus datos pasan a los campos para poder actualizar
```

```
        tblAlquileres.getSelectionModel().addListSelectionListener(e -> {
            int fila = tblAlquileres.getSelectedRow();
            if (!e.getValueIsAdjusting() && fila >= 0) {
                txtCliente.setText(String.valueOf(tblAlquileres.getValueAt(fila, 2)));
                txtMinutos.setText(String.valueOf(tblAlquileres.getValueAt(fila, 5)));
            }
        });
        // los datos son compartidos con la otra vista: al volver a esta ventana se recargan
        addWindowListener(new java.awt.event.WindowAdapter() {
            @Override
            public void windowActivated(java.awt.event.WindowEvent e) {
                cargar();
            }
        });
        setSize(900, 440);
        setLocationRelativeTo(null);
        refrescar();
    }

    /** Recarga el combo de scooters y la tabla. */
    private void cargar() {
        cmbScooter.removeAllItems();
        controlador.listarScooters().forEach(cmbScooter::addItem);
        mostrar(controlador.listar());
    }

    private void refrescar() {
        txtCliente.setText("");
        txtMinutos.setText("");
        cargar();
    }

    private void mostrar(List<Alquiler> alquileres) {
        modelo.setRowCount(0);
        alquileres.forEach(a -> modelo.addRow(new Object[]{a.getId(), a.getScooter(),
a.getCliente(),
            a.getInicio().format(FORMATO), a.getFin().format(FORMATO), a.getMinutos(),
            String.format("%.2f", a.getMonto()), a.isPagado() ? "Si" : "No"}));
    }

    /** Ejecuta la accion y, si el controller rechaza los datos, muestra el aviso. */
    private void ejecutar(Runnable accion) {
        try {
            accion.run();
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Escriba solo numeros en los campos numericos.",
"Aviso", JOptionPane.WARNING_MESSAGE);
        } catch (DateTimeParseException e) {
            JOptionPane.showMessageDialog(this, "Escriba la fecha como dd/MM/yyyy (ej.
08/10/2026).", "Aviso", JOptionPane.WARNING_MESSAGE);
        } catch (IllegalArgumentException e) {
            JOptionPane.showMessageDialog(this, e.getMessage(), "Aviso",
JOptionPane.WARNING_MESSAGE);
        }
    }

    /** Devuelve el ID de la fila elegida; si no hay fila elegida lanza un aviso. */
    private int idSeleccionado() {
        int fila = tblAlquileres.getSelectedRow();
        if (fila < 0) {
            throw new IllegalArgumentException("Seleccione una fila de la tabla.");
        }
        return (Integer) tblAlquileres.getValueAt(fila, 0);
    }

    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    @SuppressWarnings("unchecked")
    private void initComponents() {

        pnlDatos = new javax.swing.JPanel();
        lblScooter = new javax.swing.JLabel();
        cmbScooter = new javax.swing.JComboBox<>();
        lblCliente = new javax.swing.JLabel();
        txtCliente = new javax.swing.JTextField();
        lblMinutos = new javax.swing.JLabel();
        txtMinutos = new javax.swing.JTextField();
        btnRegistrar = new javax.swing.JButton();
        btnActualizar = new javax.swing.JButton();
        btnEliminar = new javax.swing.JButton();
        scrollAlquileres = new javax.swing.JScrollPane();
```

```
        tblAlquileres = new javax.swing.JTable();
        pnlBuscar = new javax.swing.JPanel();
        lblId = new javax.swing.JLabel();
        txtId = new javax.swing.JTextField();
        btnBuscar = new javax.swing.JButton();
        lblDesde = new javax.swing.JLabel();
        txtDesde = new javax.swing.JTextField();
        lblHasta = new javax.swing.JLabel();
        txtHasta = new javax.swing.JTextField();
        btnFiltrar = new javax.swing.JButton();
        btnListar = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Alquileres");

        pnlDatos.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
        lblScooter.setText("Scooter:");


        lblCliente.setText("Cliente:");

        txtCliente.setColumns(10);

        lblMinutos.setText("Minutos:");

        txtMinutos.setColumns(4);

        btnRegistrar.setText("Registrar");
        btnRegistrar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnRegistrarActionPerformed(evt);
            }
        });

        btnActualizar.setText("Actualizar");
        btnActualizar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnActualizarActionPerformed(evt);
            }
        });

        btnEliminar.setText("Eliminar");
        btnEliminar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEliminarActionPerformed(evt);
            }
        });

        pnlDatos.add(lblScooter);
        pnlDatos.add(cmbScooter);
        pnlDatos.add(lblCliente);
        pnlDatos.add(txtCliente);
        pnlDatos.add(lblMinutos);
        pnlDatos.add(txtMinutos);
        pnlDatos.add(btnRegistrar);
        pnlDatos.add(btnActualizar);
        pnlDatos.add(btnEliminar);


        scrollAlquileres.setViewportView(tblAlquileres);

        pnlBuscar.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
        lblId.setText("Buscar ID:");

        txtId.setColumns(4);

        btnBuscar.setText("Buscar");
        btnBuscar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnBuscarActionPerformed(evt);
            }
        });

        lblDesde.setText("Desde (dd/MM/yyyy):");

        txtDesde.setColumns(8);

        lblHasta.setText("Hasta:");

        txtHasta.setColumns(8);
```

```
        btnFiltrar.setText("Filtrar");
        btnFiltrar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnFiltrarActionPerformed(evt);
            }
        });

        btnListar.setText("Listar todo");
        btnListar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnListarActionPerformed(evt);
            }
        });

        pnlBuscar.add(lblId);
        pnlBuscar.add(txtId);
        pnlBuscar.add(btnBuscar);
        pnlBuscar.add(lblDesde);
        pnlBuscar.add(txtDesde);
        pnlBuscar.add(lblHasta);
        pnlBuscar.add(txtHasta);
        pnlBuscar.add(btnFiltrar);
        pnlBuscar.add(btnListar);

        getContentPane().add(pnlDatos, java.awt.BorderLayout.NORTH);
        getContentPane().add(scrollAlquileres, java.awt.BorderLayout.CENTER);
        getContentPane().add(pnlBuscar, java.awt.BorderLayout.SOUTH);

        pack();
    }// </editor-fold>//GEN-END:initComponents

    private void btnRegistrarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnRegistrarActionPerformed
        ejecutar(() -> {
            controlador.registrar((Scooter) cmbScooter.getSelectedItem(), txtCliente.getText(),
Long.parseLong(txtMinutos.getText().trim()));
            refrescar();
        });
    }//GEN-LAST:event_btnRegistrarActionPerformed

    private void btnActualizarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnActualizarActionPerformed
        ejecutar(() -> {
            controlador.actualizar(idSeleccionado(), txtCliente.getText(),
Long.parseLong(txtMinutos.getText().trim()));
            refrescar();
        });
    }//GEN-LAST:event_btnActualizarActionPerformed

    private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnEliminarActionPerformed
        ejecutar(() -> {
            controlador.eliminar(idSeleccionado());
            refrescar();
        });
    }//GEN-LAST:event_btnEliminarActionPerformed

    private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnBuscarActionPerformed
        ejecutar(() -> mostrar(controlador.buscar(Integer.parseInt(txtId.getText().trim()))));
    }//GEN-LAST:event_btnBuscarActionPerformed

    private void btnFiltrarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnFiltrarActionPerformed
        ejecutar(() -> mostrar(controlador.filtrar(LocalDate.parse(txtDesde.getText().trim(),
FECHA),
                LocalDate.parse(txtHasta.getText().trim(), FECHA))));
    }//GEN-LAST:event_btnFiltrarActionPerformed

    private void btnListarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnListarActionPerformed
        txtId.setText("");
        txtDesde.setText("");
        txtHasta.setText("");
        refrescar();
    }//GEN-LAST:event_btnListarActionPerformed

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton btnActualizar;
```

```
    private javax.swing.JButton btnBuscar;
    private javax.swing.JButton btnEliminar;
    private javax.swing.JButton btnFiltrar;
    private javax.swing.JButton btnListar;
    private javax.swing.JButton btnRegistrar;
    private javax.swing.JComboBox<com.usil.scooter.model.Scooter> cmbScooter;
    private javax.swing.JLabel lblCliente;
    private javax.swing.JLabel lblDesde;
    private javax.swing.JLabel lblHasta;
    private javax.swing.JLabel lblId;
    private javax.swing.JLabel lblMinutos;
    private javax.swing.JLabel lblScooter;
    private javax.swing.JPanel pnlBuscar;
    private javax.swing.JPanel pnlDatos;
    private javax.swing.JScrollPane scrollAlquileres;
    private javax.swing.JTable tblAlquileres;
    private javax.swing.JTextField txtCliente;
    private javax.swing.JTextField txtDesde;
    private javax.swing.JTextField txtHasta;
    private javax.swing.JTextField txtId;
    private javax.swing.JTextField txtMinutos;
    // End of variables declaration//GEN-END:variables
}
```

Archivo: src/main/java/com/usil/scooter/view/PagoFrame.java

```
package com.usil.scooter.view;

import com.usil.scooter.controller.PagoController;
import com.usil.scooter.model.Alquiler;
import com.usil.scooter.model.Pago;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.List;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

/** Vista de Pagos: aqui se paga un alquiler y se consultan los pagos. Se edita en la pestana
Design de NetBeans. */
public class PagoFrame extends javax.swing.JFrame {

    private static final DateTimeFormatter FORMATO = DateTimeFormatter.ofPattern("dd/MM HH:mm");
    private static final DateTimeFormatter FECHA = DateTimeFormatter.ofPattern("dd/MM/yyyy");

    private final PagoController controlador;
    private final DefaultTableModel modelo = new DefaultTableModel(
            new Object[]{"ID", "Alquiler", "Cliente", "Monto", "Fecha", "Tarjeta"}, 0);

    public PagoFrame(PagoController controlador) {
        this.controlador = controlador;
        initComponents();
        tblPagos.setModel(modelo);
        // los datos son compartidos con las otras vistas: al volver a esta ventana se recargan
        addWindowListener(new java.awt.event.WindowAdapter() {
            @Override
            public void windowActivated(java.awt.event.WindowEvent e) {
                cargar();
            }
        });
        setSize(860, 420);
        setLocationRelativeTo(null);
        refrescar();
    }

    /** Recarga el combo de alquileres pendientes y la tabla de pagos. */
    private void cargar() {
        cmbAlquiler.removeAllItems();
        controlador.alquileresPendientes().forEach(cmbAlquiler::addItem);
        mostrar(controlador.listar());
    }

    private void refrescar() {
        txtTarjeta.setText("");
        cargar();
    }

    private void mostrar(List<Pago> pagos) {
```

```
        modelo.setRowCount(0);
        pagos.forEach(p -> modelo.addRow(new Object[]{p.getId(), p.getAlquiler().getId(),
p.getAlquiler().getCliente(),
            String.format("%.2f", p.getMonto()), p.getFecha().format(FORMATO), p.getTarjeta()}));
    }

    /** Ejecuta la accion y, si el controller rechaza los datos, muestra el aviso. */
    private void ejecutar(Runnable accion) {
        try {
            accion.run();
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Escriba solo numeros en los campos numericos.",
"Aviso", JOptionPane.WARNING_MESSAGE);
        } catch (DateTimeParseException e) {
            JOptionPane.showMessageDialog(this, "Escriba la fecha como dd/MM/yyyy (ej.
08/10/2026).", "Aviso", JOptionPane.WARNING_MESSAGE);
        } catch (IllegalArgumentException e) {
            JOptionPane.showMessageDialog(this, e.getMessage(), "Aviso",
JOptionPane.WARNING_MESSAGE);
        }
    }

    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    @SuppressWarnings("unchecked")
    private void initComponents() {

        pnlDatos = new javax.swing.JPanel();
        lblAlquiler = new javax.swing.JLabel();
        cmbAlquiler = new javax.swing.JComboBox<>();
        lblTarjeta = new javax.swing.JLabel();
        txtTarjeta = new javax.swing.JTextField();
        btnPagar = new javax.swing.JButton();
        scrollPagos = new javax.swing.JScrollPane();
        tblPagos = new javax.swing.JTable();
        pnlBuscar = new javax.swing.JPanel();
        lblId = new javax.swing.JLabel();
        txtId = new javax.swing.JTextField();
        btnBuscar = new javax.swing.JButton();
        lblDesde = new javax.swing.JLabel();
        txtDesde = new javax.swing.JTextField();
        lblHasta = new javax.swing.JLabel();
        txtHasta = new javax.swing.JTextField();
        btnFiltrar = new javax.swing.JButton();
        btnListar = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Pagos");

        pnlDatos.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
        lblAlquiler.setText("Alquiler pendiente:");


        lblTarjeta.setText("Tarjeta (16 digitos):");

        txtTarjeta.setColumns(14);

        btnPagar.setText("Pagar");
        btnPagar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnPagarActionPerformed(evt);
            }
        });

        pnlDatos.add(lblAlquiler);
        pnlDatos.add(cmbAlquiler);
        pnlDatos.add(lblTarjeta);
        pnlDatos.add(txtTarjeta);
        pnlDatos.add(btnPagar);


        scrollPagos.setViewportView(tblPagos);

        pnlBuscar.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
        lblId.setText("Buscar ID:");

        txtId.setColumns(4);

        btnBuscar.setText("Buscar");
        btnBuscar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
                btnBuscarActionPerformed(evt);
            }
        });

        lblDesde.setText("Desde (dd/MM/yyyy):");

        txtDesde.setColumns(8);

        lblHasta.setText("Hasta:");

        txtHasta.setColumns(8);

        btnFiltrar.setText("Filtrar");
        btnFiltrar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnFiltrarActionPerformed(evt);
            }
        });

        btnListar.setText("Listar todo");
        btnListar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnListarActionPerformed(evt);
            }
        });

        pnlBuscar.add(lblId);
        pnlBuscar.add(txtId);
        pnlBuscar.add(btnBuscar);
        pnlBuscar.add(lblDesde);
        pnlBuscar.add(txtDesde);
        pnlBuscar.add(lblHasta);
        pnlBuscar.add(txtHasta);
        pnlBuscar.add(btnFiltrar);
        pnlBuscar.add(btnListar);

        getContentPane().add(pnlDatos, java.awt.BorderLayout.NORTH);
        getContentPane().add(scrollPagos, java.awt.BorderLayout.CENTER);
        getContentPane().add(pnlBuscar, java.awt.BorderLayout.SOUTH);

        pack();
    }// </editor-fold>//GEN-END:initComponents

    private void btnPagarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnPagarActionPerformed
        ejecutar(() -> {
            Alquiler alquiler = (Alquiler) cmbAlquiler.getSelectedItem();
            if (alquiler == null) {
                throw new IllegalArgumentException("No hay alquileres pendientes de pago.");
            }
            controlador.pagar(alquiler.getId(), txtTarjeta.getText());
            refrescar();
            JOptionPane.showMessageDialog(this, "Pago aprobado.");
        });
    }//GEN-LAST:event_btnPagarActionPerformed

    private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnBuscarActionPerformed
        ejecutar(() -> mostrar(controlador.buscar(Integer.parseInt(txtId.getText().trim()))));
    }//GEN-LAST:event_btnBuscarActionPerformed

    private void btnFiltrarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnFiltrarActionPerformed
        ejecutar(() -> mostrar(controlador.filtrar(LocalDate.parse(txtDesde.getText().trim(),
FECHA),
                LocalDate.parse(txtHasta.getText().trim(), FECHA))));
    }//GEN-LAST:event_btnFiltrarActionPerformed

    private void btnListarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_btnListarActionPerformed
        txtId.setText("");
        txtDesde.setText("");
        txtHasta.setText("");
        refrescar();
    }//GEN-LAST:event_btnListarActionPerformed

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton btnBuscar;
    private javax.swing.JButton btnFiltrar;
    private javax.swing.JButton btnListar;
```

```
    private javax.swing.JButton btnPagar;
    private javax.swing.JComboBox<com.usil.scooter.model.Alquiler> cmbAlquiler;
    private javax.swing.JLabel lblAlquiler;
    private javax.swing.JLabel lblDesde;
    private javax.swing.JLabel lblHasta;
    private javax.swing.JLabel lblId;
    private javax.swing.JLabel lblTarjeta;
    private javax.swing.JPanel pnlBuscar;
    private javax.swing.JPanel pnlDatos;
    private javax.swing.JScrollPane scrollPagos;
    private javax.swing.JTable tblPagos;
    private javax.swing.JTextField txtDesde;
    private javax.swing.JTextField txtHasta;
    private javax.swing.JTextField txtId;
    private javax.swing.JTextField txtTarjeta;
    // End of variables declaration//GEN-END:variables
}
```

Volver al índice

Paso 10. Main y ejecución

Main abre el menú principal. Para ejecutar: clic derecho en el proyecto > Run (o F6).

Archivo: src/main/java/com/usil/scooter/Main.java

```
package com.usil.scooter;

import com.usil.scooter.view.MainFrame;

public class Main {

    public static void main(String[] args) {
        java.awt.EventQueue.invokeLater(() -> new MainFrame().setVisible(true));
    }
}
```

Volver al índice

Paso 11. Probar la aplicación

Acción	Resultado esperado
Registrar un scooter: Xiaomi, Pro 2, tarifa 0.50	Aparece en la tabla de scooters.
Registrar un alquiler de 30 minutos para ese scooter	Monto 15,00 (30 × 0.50).
Pagar con una tarjeta de 4 dígitos	Aviso: tarjeta inválida, no se crea el pago.
Pagar con una tarjeta de 16 dígitos	Pago aprobado; el alquiler queda como pagado.
Actualizar el alquiler ya pagado	Aviso: un alquiler pagado no se puede modificar.
Eliminar el scooter que tiene alquileres	Aviso: el scooter tiene alquileres.
Filtrar alquileres con el rango de hoy a hoy	Aparecen los alquileres de hoy.

Volver al índice